

Тема: Наследование.

Задача «Тамагочи»

Тамагочи - игрушка, виртуальный домашний питомец. Идея создания принадлежала компании Bandai — третьему по величине производителю игрушек планеты. Слово произошло от двух слов: японского тамаго (яп. たまご, «яйцо») и англицизма уотти (яп. ウオッチ, от «Watch» — часы).

Создать проект «Tamagotchi». В проекте создать базовый класс Pets (домашние животные). В классе прописать все общие поля, свойства и методы классов Cat и Dog. Классы Cat и Dog сделать наследниками класса Pets. Повторяющиеся блоки кода/методы убрать или переопределить. Придумать ещё классы наследники и свой сценарий поведения животных.

Ниже пример класса Cat без наследования.

В проекте создать класс «Cat». Реализовать методы Eat() - «покушать» и Meow() - «помяукать». Кошки могут есть к примеру фрукты – яблоки и апельсины, – а у фруктов есть некоторое количество здоровья. Когда кошки едят фрукты, то им добавляются единицы здоровья и в консоль выводится информация о том что кошка съела и сколько добавилось здоровья.

Выполняйте задание по инструкции.

Шаг 1. Создадим проект «Tamagotchi». В проект добавим класс «Cat».

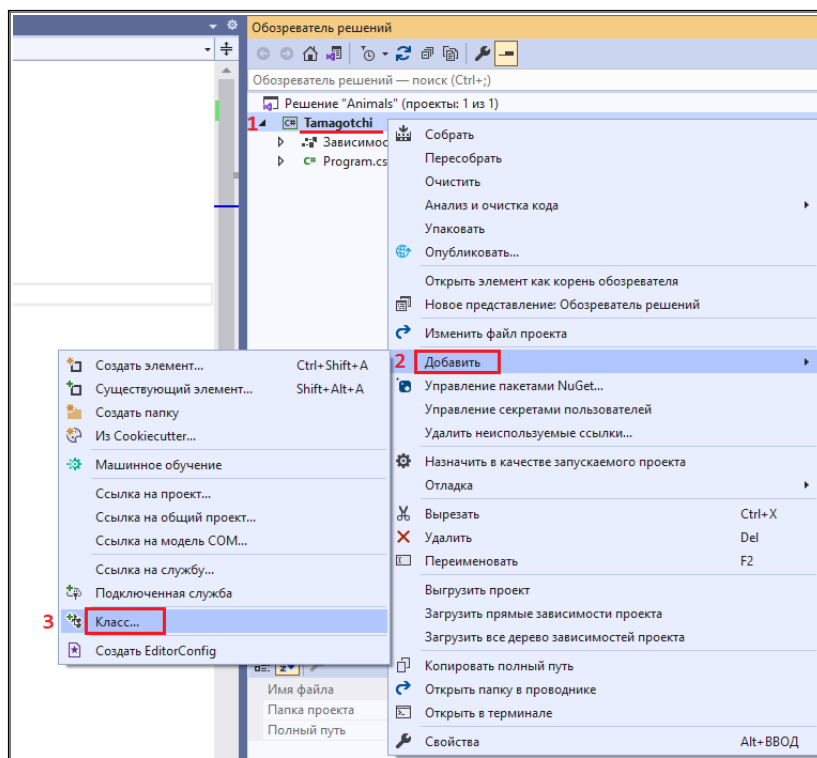


Рисунок 1- Создать класс Cat

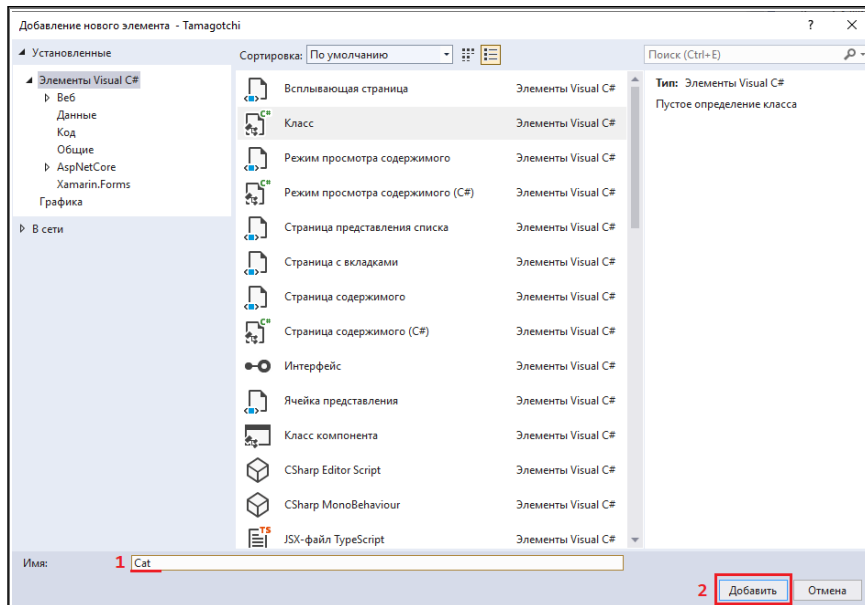


Рисунок 2- Задать классу имя

В обозревателе решений добавлен новый класс Cat и текст кода.

```
using System;
using System.Collections.Generic;
using System.Text;

namespace Tamagotchi
{
    class Cat
    {
    }
}
```

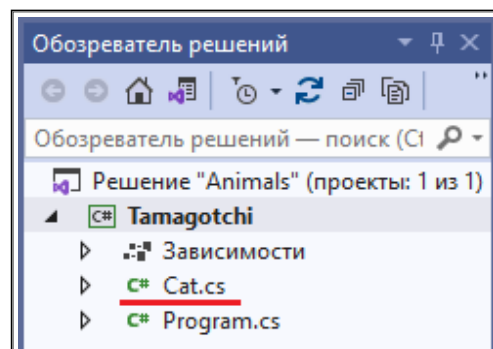


Рисунок 3 — Иерархия проекта

В классе Program создадим экземпляр класса Cat. Запустим проект.

```
using System;

namespace Tamagotchi
{
    class Program
    {
        static void Main(string[] args)
        {
            Cat cat = new Cat(); // создали кошку
        }
    }
}
```

Проект запускается, но в консоль пока ничего не выводится.

Шаг 2. Допишем в класс Cat метод Meow()

```
using System;
using System.Collections.Generic;
using System.Text;

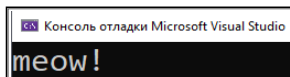
namespace Tamagotchi
{
    class Cat // класс Кошка
    {
        public void Meow() // кошка может мяукать
        {
            Console.WriteLine("meow! ");
        }
    }
}
```

В классе Program вызовем метод Meow().

```
using System;

namespace Tamagotchi
{
    class Program
    {
        static void Main(string[] args)
        {
            Cat cat = new Cat(); // создали кошку
            cat.Meow(); // помяукали
        }
    }
}
```

Запустим проект. В консоль выводится: meow!



Шаг 3. Добавим структуры Apple и Orange. В класс Cat добавим метод Eat()

```
using System;
using System.Collections.Generic;
using System.Text;

namespace Tamagotchi
{
    struct Apple
    { // структура Яблоко
        public int health;
    }
}
```

```

struct Orange
{ // структура Апельсин
    public int health;
}

class Cat // класс Кошка
{
    public void Meow() // кошка может мяукать
    {
        Console.WriteLine("meow! ");
    }

    public void Eat(Apple a) // кошка может есть яблоко
    {
        Console.WriteLine($"Cat eats apple. {a.health} hp.\n");
    }
}

```

В классе Program создадим объект Apple и вызовем метод Eat().

```

using System;

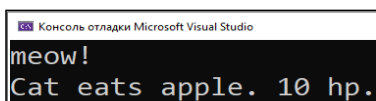
namespace Tamagotchi
{
    class Program
    {
        static void Main(string[] args)
        {
            Cat cat = new Cat(); // создали кошку
            cat.Meow(); // помяукали
            Apple apple = new Apple { health = 10 }; // создали яблоко
            cat.Eat(apple); // кошка съела яблоко
        }
    }
}

```

Запустим проект. В консоль выводится:

meow!

Cat eats apple. 10hp.



Всё, кошка съела яблоко, в котором было 10 единиц здоровья. А теперь попробуем скормить кошке апельсин. По идее она должна уметь есть фрукты, и апельсины тоже.

```

using System;

namespace Tamagotchi
{

```

```

class Program
{
    static void Main(string[] args)
    {
        Cat cat = new Cat(); // создали кошку
        cat.Meow(); // помяукали
        Apple apple = new Apple { health = 10 }; // создали яблоко
        cat.Eat(apple); // кошка съела яблоко
        Orange orange = new Orange { health = 5 }; //создали апельсин
        cat.Eat(orange); //Program.cs(14, 21, 14, 27): error CS1503:
// Аргумент 1: не удается преобразовать из "Tamagotchi.Orange" в
//"Tamagotchi.Apple".
    }
}

```

Появляется ошибка: нет метода «поесть» для апельсина. Определим его, продублировав метод яблока в классе Cat:

```

class Cat // класс Кошка
{
    public void Meow() // кошка может мяукать
    {
        Console.WriteLine("meow! ");
    }

    public void Eat(Apple a) // кошка может есть яблоко
    {
        Console.WriteLine($"Cat eats apple. {a.health} hp.");
    }

    public void Eat(Orange o) // кошка может есть апельсин
    {
        Console.WriteLine($"Cat eats orange. {o.health} hp.");
    }
}

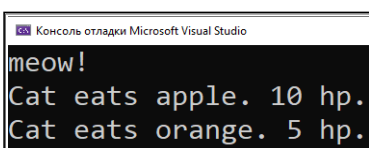
```

Запустим проект. В консоль выводится:

meow!

Cat eats apple. 10hp.

Cat eats orange. 5hp.



Всё, апельсин тоже съели. Но получилось дублирование кода. Заменим в коде структуры

Apple и Orange на общую структуру Fruit с полями **type** и **health**. И в классе Cat оставим один метод Eat():

```
using System;
using System.Collections.Generic;
using System.Text;

namespace Tamagotchi
{
    struct Fruit
    { // структура Фрукт
        public string type;
        public int health;
    }

    class Cat // класс Кошка
    {
        public void Meow() // кошка может мяукать
        {
            Console.WriteLine("meow! ");
        }

        public void Eat(Fruit f) // кошка может есть фрукт
        {
            Console.WriteLine($"Cat eats {f.type}. {f.health} hp.");
        }
    }
}
```

Изменим строки кода в классе Program, где создавали яблоко и апельсин на создать фрукт:

```
using System;

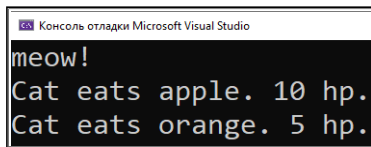
namespace Tamagotchi
{
    class Program
    {
        static void Main(string[] args)
        {
            Cat cat = new Cat(); // создали кошку
            cat.Meow(); // помяукали
            // создали фрукт
            Fruit fruitApple = new Fruit { type = "apple", health = 10 };
            cat.Eat(fruitApple); // кошка съела яблоко
            // создали фрукт
            Fruit fruitOrange = new Fruit { type = "orange", health = 5 };
            cat.Eat(fruitOrange); // кошка съела апельсин
        }
    }
}
```

Запустим проект. Все работает! В консоль снова выводится:

meow!

Cat eats apple. 10hp.

Cat eats orange. 5hp.



```
Консоль отладки Microsoft Visual Studio
meow!
Cat eats apple. 10 hp.
Cat eats orange. 5 hp.
```

Добавим в класс `Cat` поле `name` - имя и поле `hunger` - голод. На данном этапе обучения не будем соблюдать инкапсуляцию и поля будут доступны всем. В описание полей мы добавим модификатор `public`.

Изменим код в методе `Eat()`. Когда выполняется метод `Eat()` будет изменяться значение поля `hunger` и выводится сообщение о том, что кошка съела и сколько здоровья добавилось. В сообщение добавим имя кошки. Данные изменения внесите в код самостоятельно.

В классе `Program` давайте создадим метод «покормить», `DoMeal`. В метод мы будем передавать животное кошку и некоторый фрукт, при этом сам метод не будет знать, что это за фрукт. Он просто вызовет метод «поесть» для этого фрукта. И, по идее, вывод в консоль измениться не должен.

```
using System;
```

```
namespace Tamagotchi
```

```
{
    class Program
    {
        static void Main(string[] args)
        {
            // создали кошку
            Cat cat = new Cat { name = "Pushok", hunger = 5 };
            cat.Meow(); // помяукали
            // создали фрукт яблоко
            Fruit fruitApple = new Fruit { type = "apple", health = 10 };
            // создали фрукт апельсин
            Fruit fruitOrange = new Fruit { type = "orange", health = 5 };
            DoMeal(cat, fruitApple); // покормили кошку яблоком
            DoMeal(cat, fruitApple); // покормили кошку яблоком
            DoMeal(cat, fruitOrange); // покормили кошку апельсином
        }

        static void DoMeal(Cat c, Fruit f)
        {
            c.Eat(f); // кошка {имя кошки} съела фрукт
        }
    }
}
```

```
}  
}
```

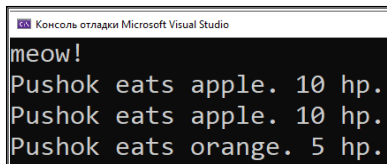
Запустим проект. Все работает! В консоль выводится:

meow!

Pushok eats apple. 10 hp.

Pushok eats apple. 10 hp.

Pushok eats orange. 5 hp.



В классе `Program` придумать и реализовать алгоритм: Если кошка голодная, например поле `hunger` (голод) будет меньше установленного значения, то кошка мяукает и её нужно покормить. Если поле `hunger` (голод) становится больше или равно установленного значения, то нужно прекратить кормить кошку и она идёт играть, прыгать. Тогда в класс `Cat` добавьте методы `Play()` и `Run()`. При выполнении этих методов кошка теряет каллории и снова начинает мяукать и её нужно кормить. Если кошку за определенное время не покормили и `hunger` (голод) становится меньше 5 или равно 0, то кошка погибает. Подумайте как можно использовать метод `ждать` со значением случайное число.

Для закрепления материала аналогично классу `Cat` реализуйте класс `Dog`.

Создать базовый класс `Pets` (домашние животные). В классе прописать все общие поля, свойства и методы классов `Cat` и `Dog`. Классы `Cat` и `Dog` сделать наследниками класса `Pets`. Повторяющиеся блоки кода/методы убрать или переопределить. Придумать ещё классы наследники.